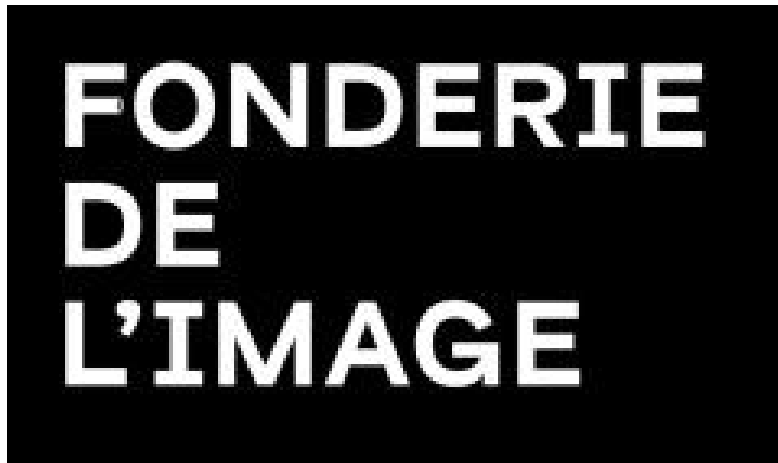




RAPPORT TECHNIQUE

Mise en Place et Optimisation d'une Infrastructure Data sur le Cloud

Collecte et stockage de données avec Python

Auteur : Lyna Lasla

Encadreur : M. Antoine Lucsko

Classe : AIA01

Date : Mai 2026

1. Introduction et contexte

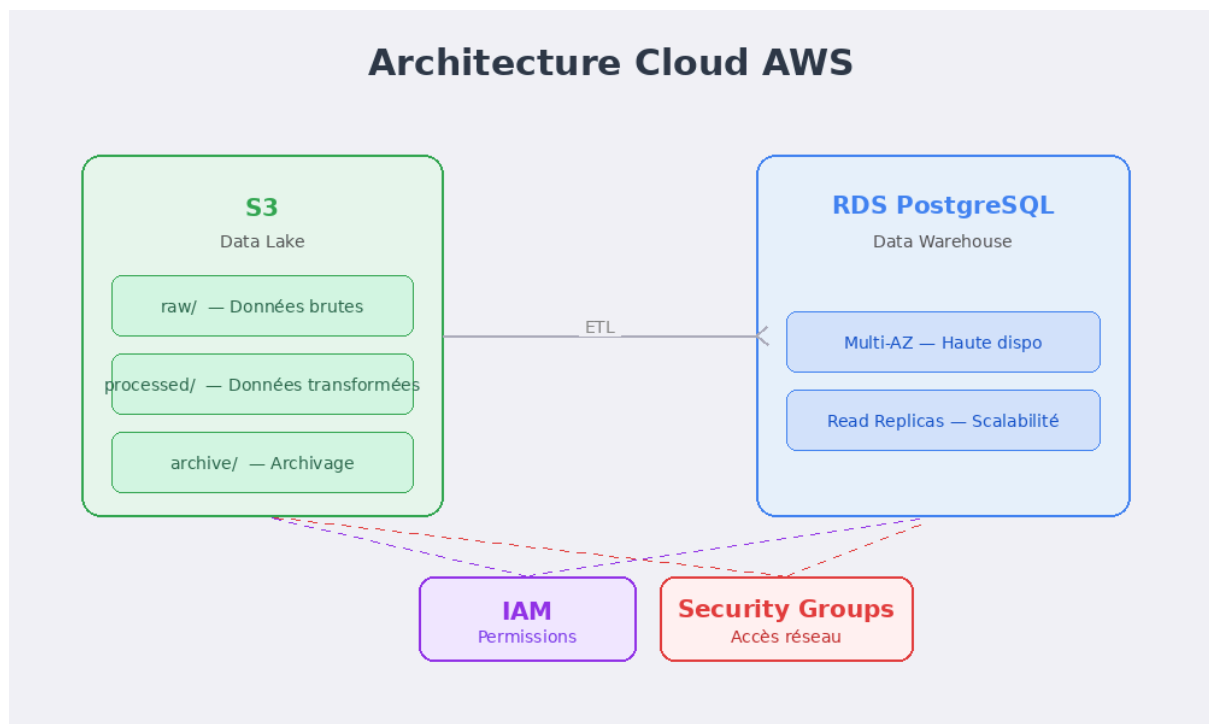
Ce projet consiste à concevoir une architecture Data moderne permettant de collecter, transformer, stocker et visualiser des données d'offres d'emploi Data. L'objectif principal est de reproduire un pipeline ETL proche des standards industriels, en utilisant une infrastructure Cloud AWS et des outils de Data Engineering.

2. Architecture Cloud AWS

L'architecture repose sur plusieurs services AWS :

- AWS S3 utilisé comme Data Lake pour stocker les données brutes et transformées.
- AWS RDS PostgreSQL utilisé comme Data Warehouse analytique.
- IAM pour la gestion des permissions.
- Security Groups pour sécuriser les accès réseau.

Cette architecture permet une séparation claire entre stockage brut et stockage analytique.



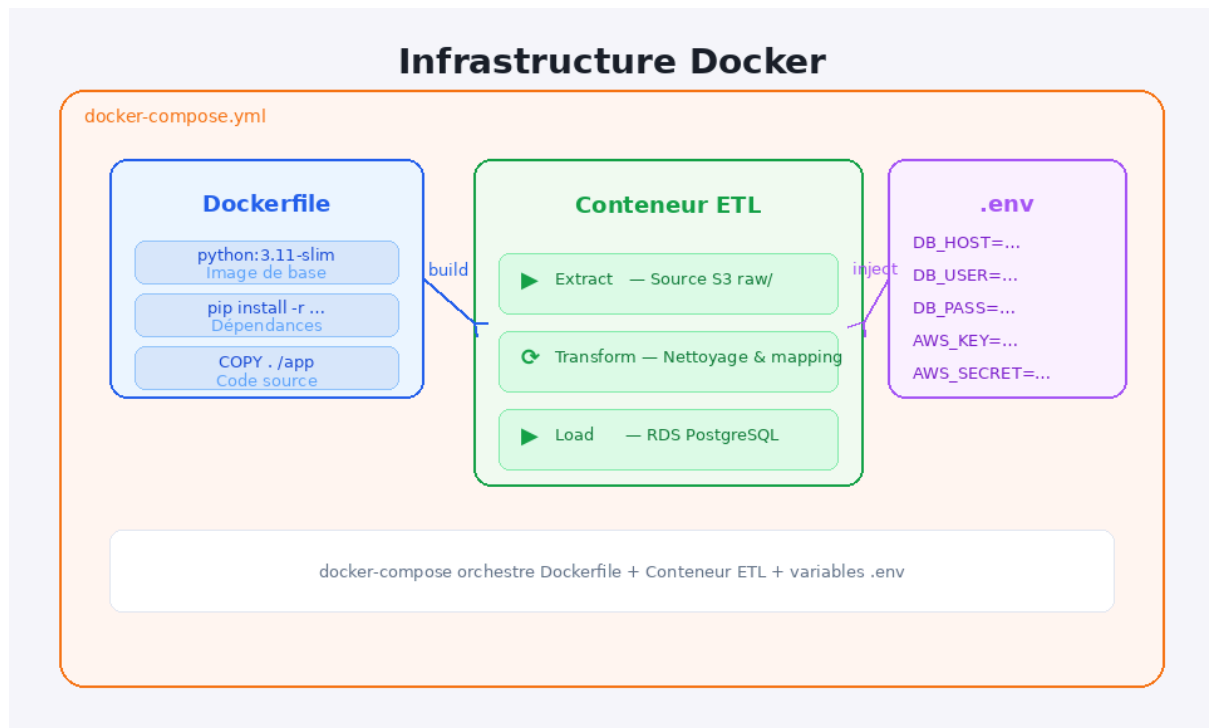
3. Infrastructure Docker

Le pipeline ETL est exécuté dans un conteneur Docker afin de garantir la reproductibilité du projet.

Docker permet de standardiser l'environnement d'exécution et simplifie le déploiement.

Le projet utilise :

- Dockerfile pour construire l'environnement Python
- docker-compose pour orchestrer les services
- Variables d'environnement via fichier .env



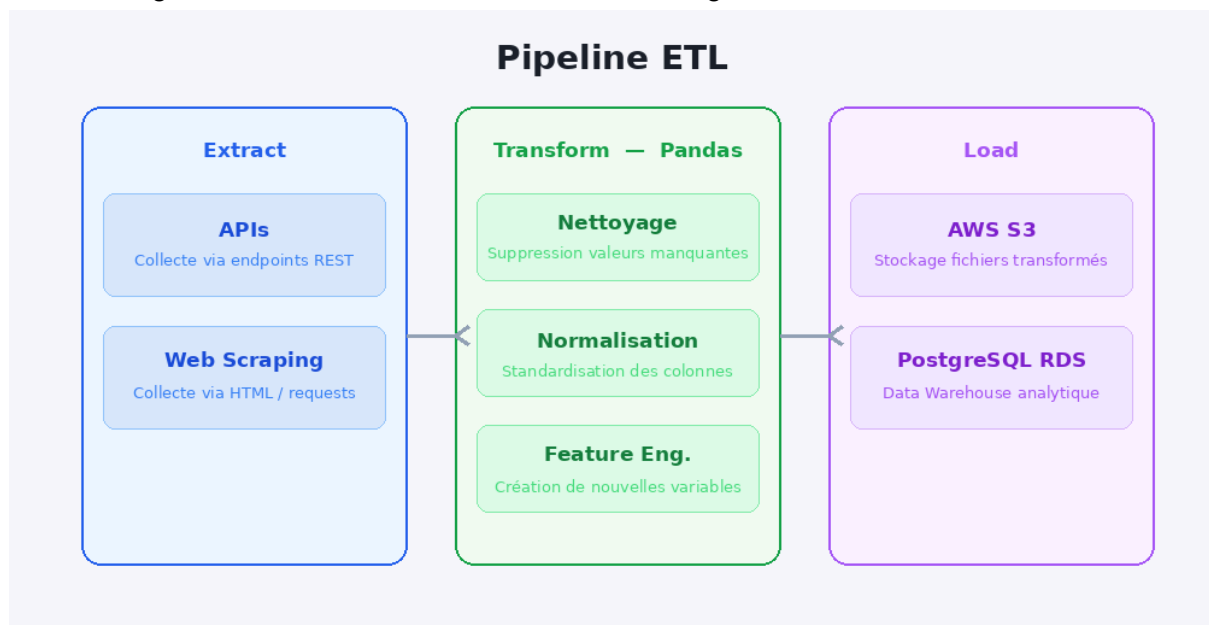
4. Pipeline ETL

Le pipeline ETL se compose de trois étapes principales :

Extraction : collecte de données depuis APIs et scraping web.

Transformation : nettoyage des données avec Pandas, suppression des valeurs manquantes, normalisation des colonnes et création de nouvelles variables analytiques.

Load : chargement des données vers AWS S3 et PostgreSQL RDS.



5. Analyse des données

Plusieurs analyses statistiques ont été réalisées :

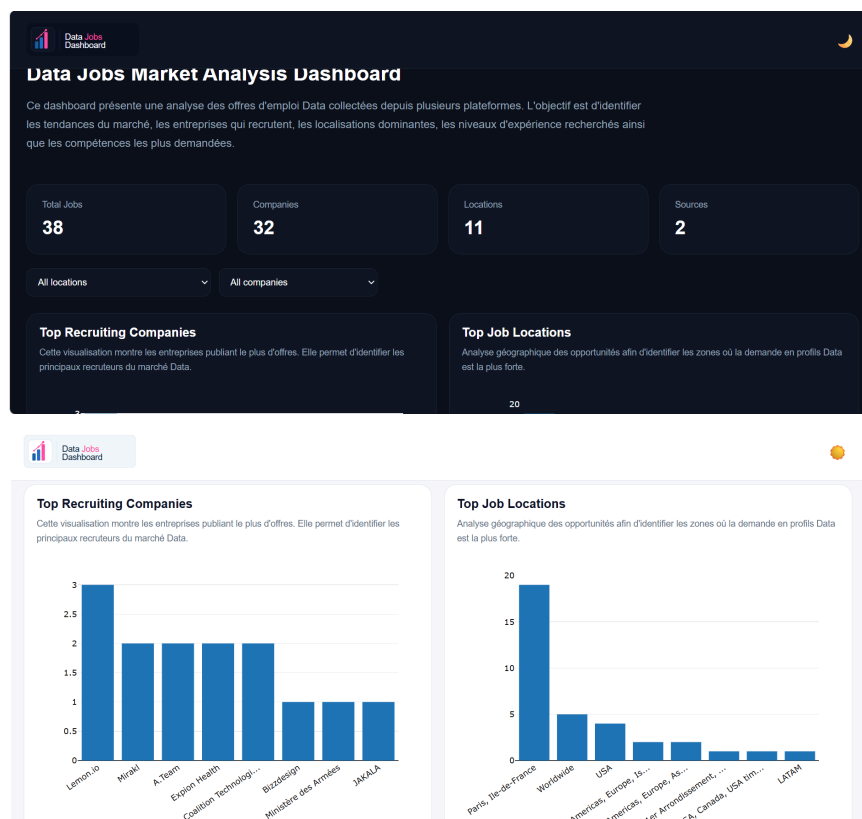
- Répartition des entreprises qui recrutent
- Analyse géographique des offres
- Répartition des niveaux de postes (junior, mid, senior)
- Détection des compétences les plus demandées (Python, SQL, AI)
- Analyse des sources de données

6. Dashboard interactif

Le dashboard interactif a été développé en HTML, CSS, JavaScript et Plotly.

Il permet :

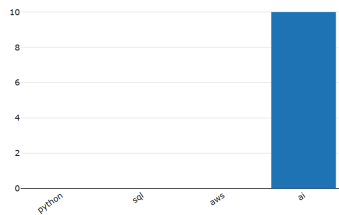
- l’affichage de KPIs dynamiques
- des filtres interactifs par ville et entreprise
- plusieurs graphiques interactifs
- un mode Dark/Light moderne Le dashboard permet une visualisation claire et interactive des données collectées.





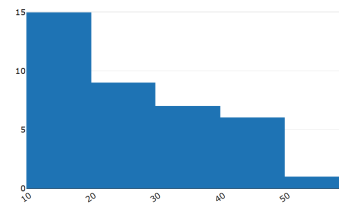
Skills Detection

Détection des compétences mentionnées dans les titres des offres afin d'identifier les technologies les plus visibles.



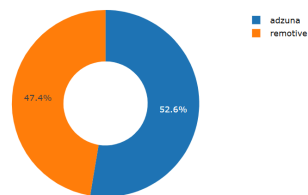
Job Title Complexity

Distribution de la longueur des intitulés de poste. Cette analyse aide à comprendre la complexité et la spécialisation des rôles proposés.



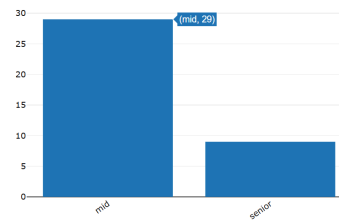
Data Sources Distribution

Répartition des offres selon les plateformes utilisées pour la collecte des données.



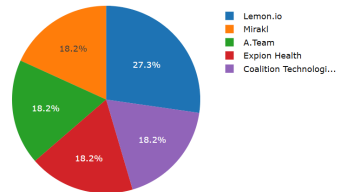
Experience Levels

Comparaison des niveaux d'expérience demandés dans les offres : junior, mid et senior.



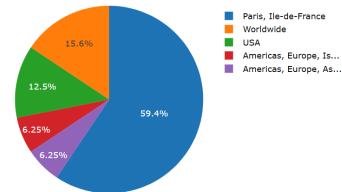
Top Companies Share

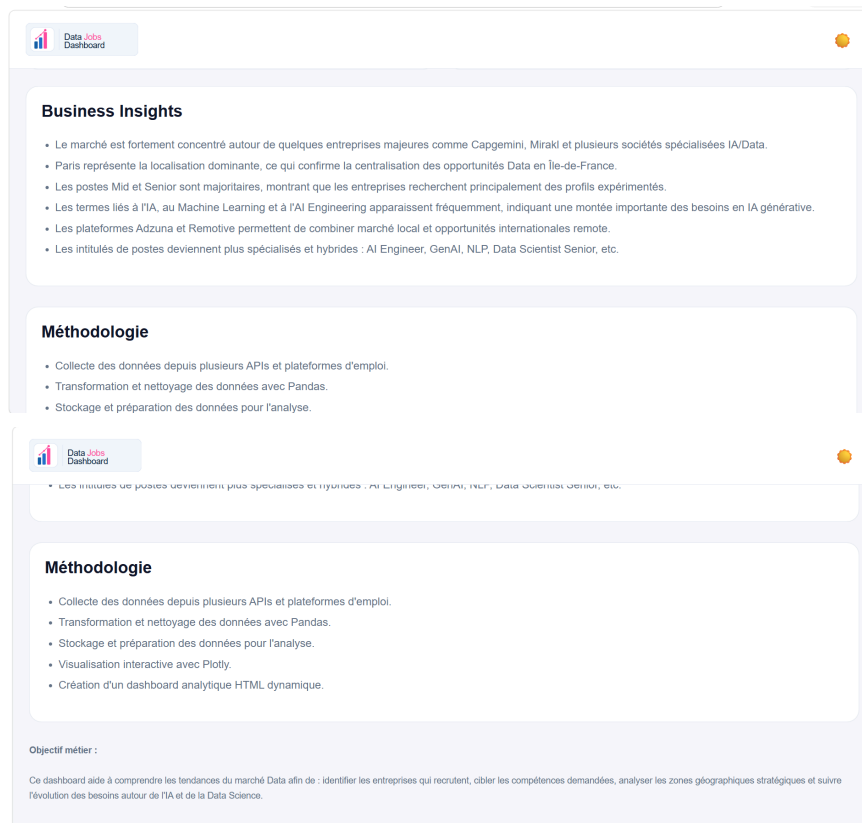
Part des principales entreprises dans l'ensemble des offres collectées.



Top Locations Share

Répartition des offres entre les principales zones géographiques.





7. Gestion de projet & GitHub

Le projet est structuré de manière modulaire :

projet-data/

```

|
|— aws/
|   |— iam_policy.json
|   |— rds_schema.sql
|   |— s3_config.md
|
|— dashboard/
|   |— app.py
|   |— dashboard.html
|   |— logo-dark.svg
|   |— logo-light.svg
|
|— data/
|   |— processed/
|   |   |— jobs_clean.csv
|   |
|   |— raw/
|   |   |— api_jobs.csv
|   |   |— scrape_jobs.csv
|
|— etl/
|   |— check_rds.py

```

```
|
| |— from_s3.py
| |— load_to_rds.py
| |— test_rds.py
| |— to_s3.py
|
|— notebooks/
|   |— analysis.ipynb
|
|— scripts/
|   |— api.py
|   |— main.py
|   |— scrape.py
|   |— transform.py
|
|— Dockerfile
|— docker-compose.yml
|— requirements.txt
|— .env
|— README.md
```

Cette organisation améliore la lisibilité, la maintenance et la réutilisation du code.

8. Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées :

- gestion des credentials AWS
- connexion sécurisée à PostgreSQL RDS
- configuration Docker et variables d'environnement
- nettoyage de données issues de plusieurs sources

9. Conclusion

Ce projet démontre la mise en place d'une architecture Data Cloud complète, intégrant ingestion, transformation, stockage et visualisation des données. Il reproduit les principales étapes d'un pipeline Data Engineering professionnel et met en œuvre des technologies largement utilisées dans l'industrie.

Schéma simplifié de l'architecture

